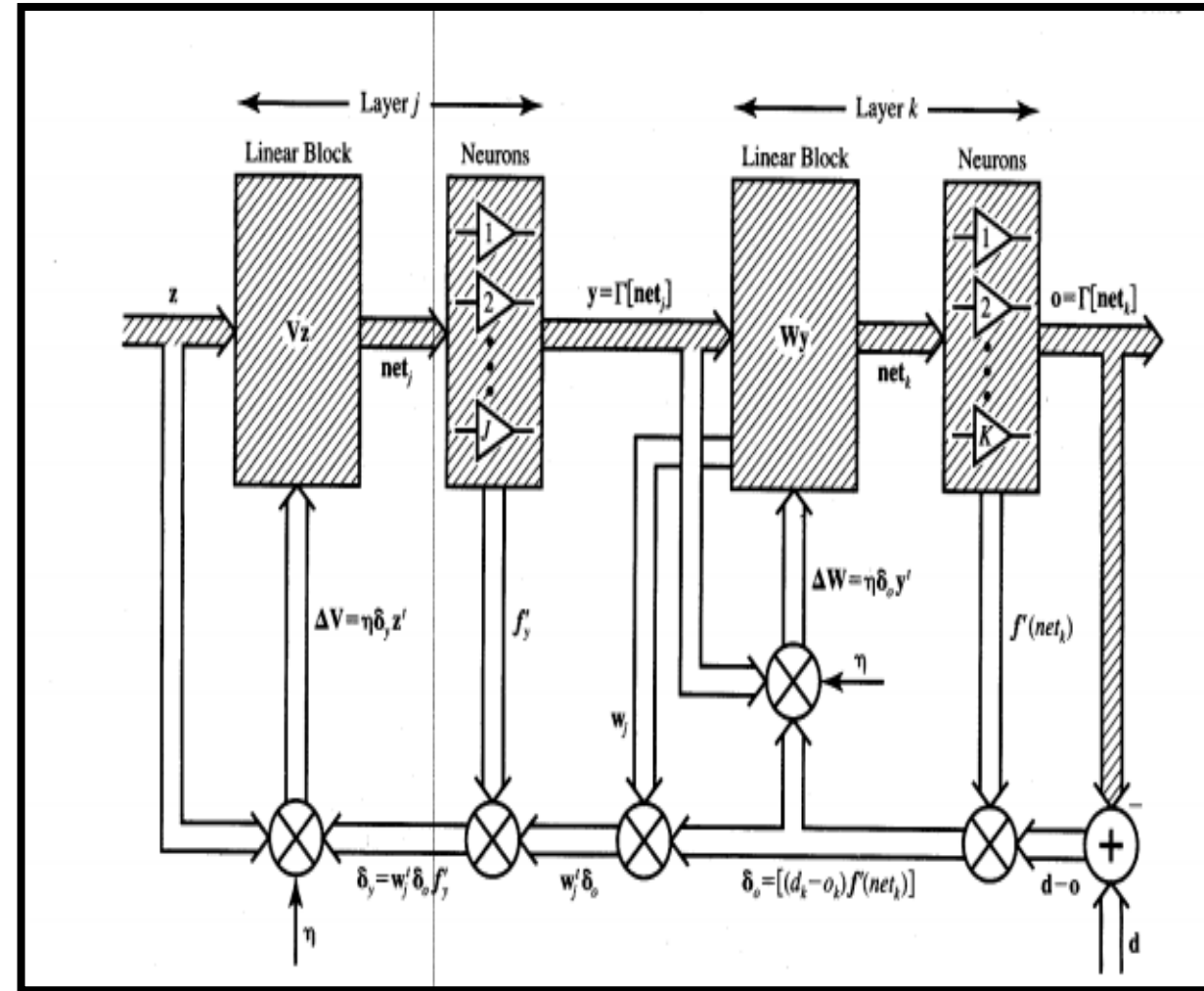
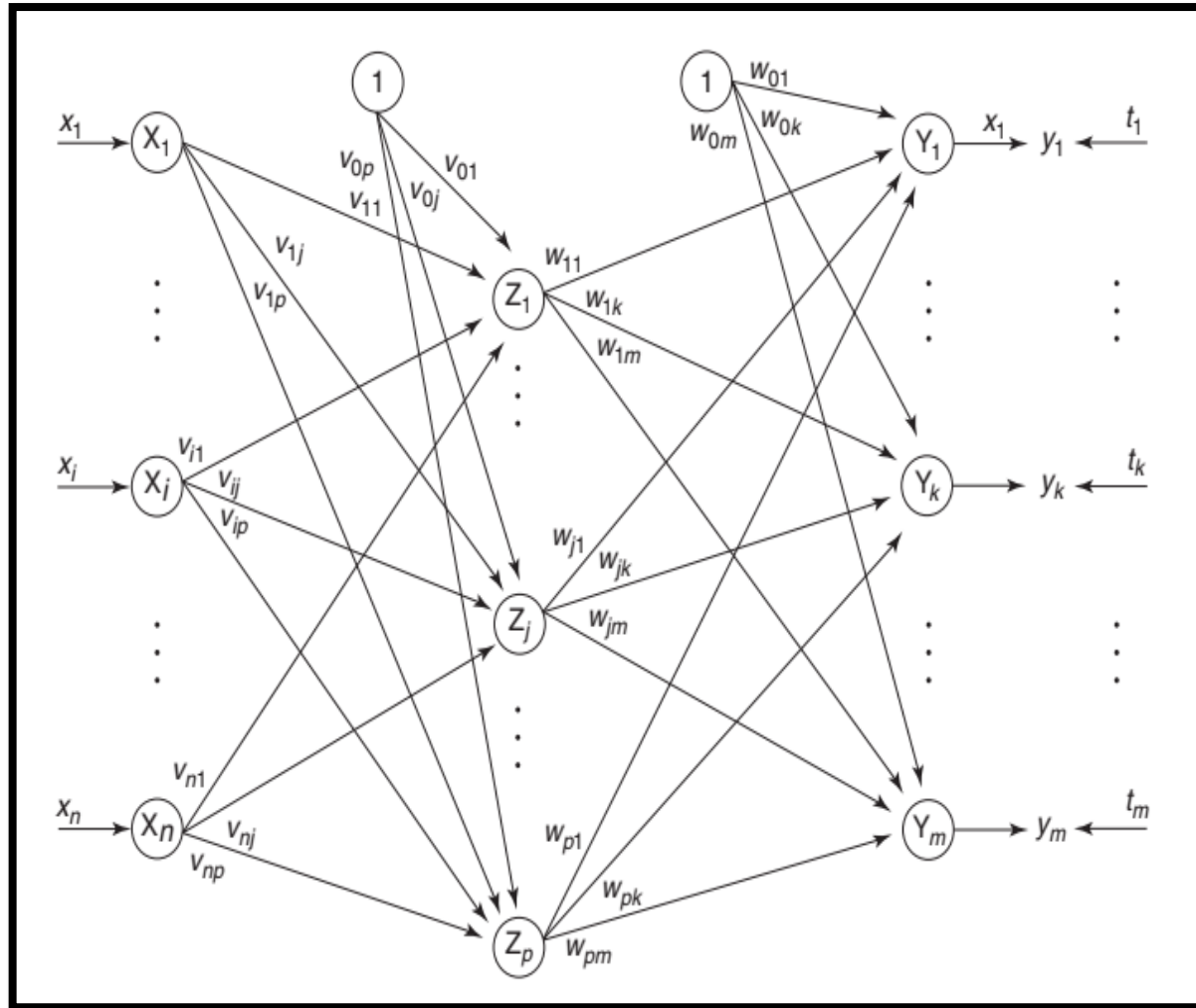


Error Back Propagation Network

Architecture of Back Propagation Network



- Forward Pass → Input signal moves forward from the network input to the output.
- Backward Pass → The calculated error signals propagate backward through the network. Weight adjustment is carried out.
- The activation function could be any function which increases monotonically and is also differentiable.

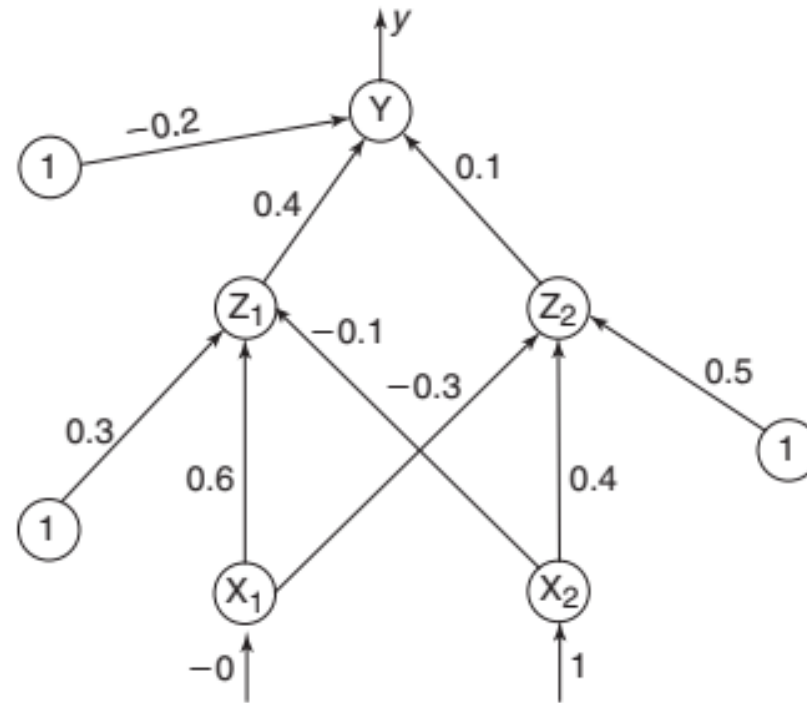
- This learning algorithm is applied to multilayer feed-forward networks consisting of processing elements with continuous differentiable activation functions.
- The networks associated with back-propagation learning algorithm are also called back-propagation networks (BPNs).
- For a given set of training input-output pair, this algorithm provides a procedure for changing the weights in a BPN to classify the given input patterns correctly.
- The basic concept for this weight update algorithm is simply the gradient-descent method with differentiable units.

- This is a method where the error is propagated back to the hidden unit.
- The aim of the neural network is to train the net to achieve a balance between the net's ability to respond (memorization) and its ability to give reasonable responses to the input that is similar but not identical to the one that is used in training (generalization).
- The back-propagation algorithm is different from other networks in respect to the process by which the weights are calculated during the learning period of the network.
- The general difficulty with the multilayer perceptron is calculating the weights of the hidden layers in an efficient way that would result in a very small or zero output error..

- When the hidden layers are increased the network training becomes more complex.
- To update weights, the error must be calculated.
- The error, which is the difference between the actual (calculated) and the desired (target) output, is easily measured at the output layer.
- At the hidden layers, there is no direct information of the error. Therefore, other techniques should be used to calculate an error at the hidden layer, which will cause minimization of the output error.

- The training of the BPN is done in three stages – the feed-forward of the input training pattern, the calculation and back-propagation of the error, and up-dation of weights.
- The testing of the BPN involves the computation of feed-forward phase only. Even though the training is very slow, once the network is trained it can produce its outputs very rapidly.

Numerical for Backpropagation Network



FLOWCHART EBPA TRAINING

x = input training vector ($x_1, \dots, x_i, \dots, x_n$)

t = target output vector ($t_1, \dots, t_k, \dots, t_m$)

α = learning rate parameter

x_i = input unit i . (Since the input layer uses identity activation function, the input and output signals here are same.)

v_{0j} = bias on j th hidden unit

w_{0k} = bias on k th output unit

z_j = hidden unit j . The net input to z_j is

$$z_{inj} = v_{0j} + \sum_{i=1} x_i v_{ij}$$

and the output is

$$z_j = f(z_{inj})$$

y_k = output unit k . The net input to y_k is

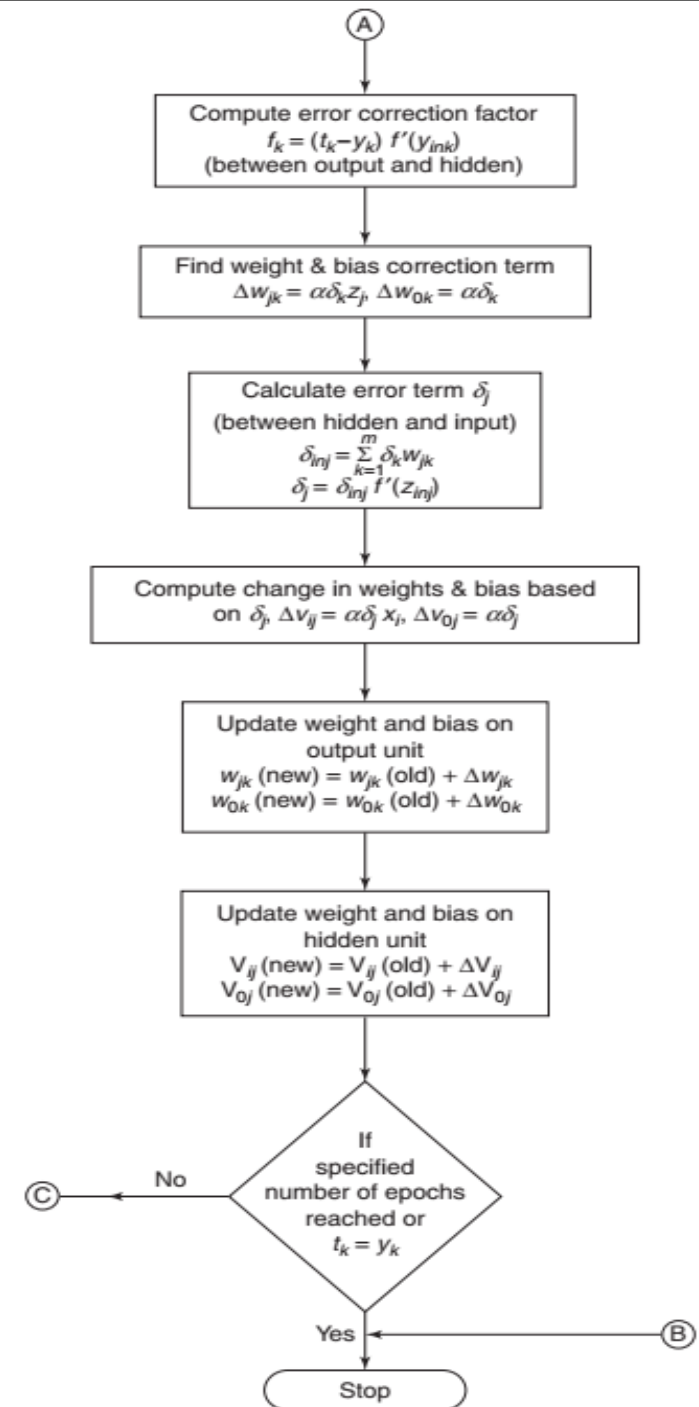
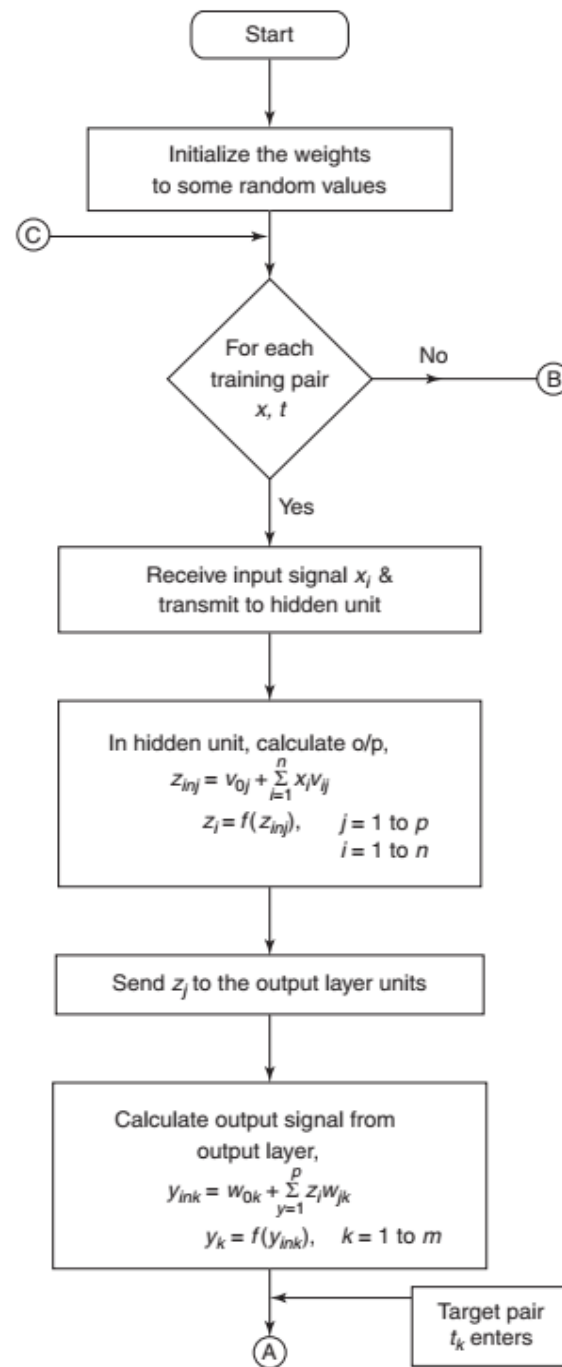
$$y_{ink} = w_{0k} + \sum_{j=1}^p z_j w_{jk}$$

and the output is

$$y_k = f(y_{ink})$$

δ_k = error correction weight adjustment for w_{jk} that is due to an error at output unit y_k , which is back-propagated to the hidden units that feed into unit y_k

δ_j = error correction weight adjustment for v_{ij} that is due to the back-propagation of error to the hidden unit z_j .



EBPA – TRAINING ALGORITHM

Step 0: Initialize weights and learning rate (take some small random values).

Step 1: Perform Steps 2–9 when stopping condition is false.

Step 2: Perform Steps 3–8 for each training pair.

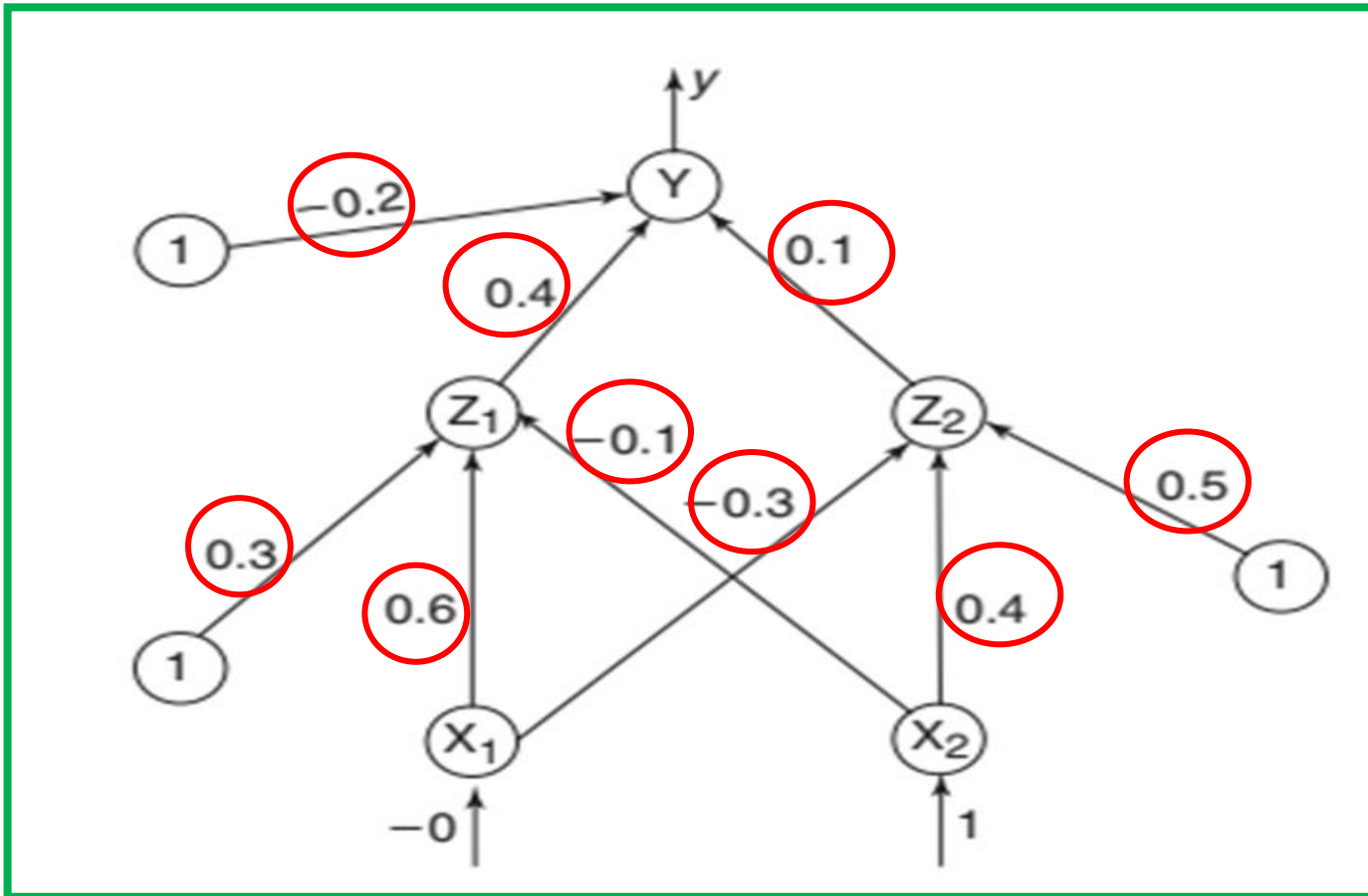
- $\alpha = 0.25$

- Input pattern
- $[0 \ 1]$

- Target
- $[1]$

- Binary sigmoidal activation function.

- $\lambda = 1$



Feed-forward phase (Phase I):

Step 3: Each input unit receives input signal x_i and sends it to the hidden unit ($i = 1$ to n).

Step 4: Each hidden unit z_j ($j = 1$ to p) sums its weighted input signals to calculate net input:

$$z_{inj} = v_{0j} + \sum_{i=1}^n x_i v_{ij}$$

Calculate output of the hidden unit by applying its activation functions over z_{inj} (sigmoidal activation function):

$$z_j = f(z_{inj})$$

and send the output signal from the hidden unit to the input of output layer units.

- Calculate net input

- $Z_{in1} = (x_1 * w_{11}) + (x_2 * w_{21}) + b_{w01}$

- $Z_{in2} = (x_1 * w_{12}) + (x_2 * w_{22}) + b_{w02}$

- Calculate output for Hidden Layer nodes

- $Z_1(f(Z_{in1}))$

- $Z_2(f(Z_{in2}))$

- $Z_{in1} = 0.2$

- $Z_{in2} = 0.9$

- $Z_1(f(Z_{in1})) = 0.5498$

- $Z_2(f(Z_{in2})) = 0.7109$

Step 5: For each output unit y_k ($k = 1$ to m), calculate the net input:

$$y_{ink} = w_{0k} + \sum_{j=1}^p z_j w_{jk}$$

and apply the activation function to compute output signal

$$y_k = f(y_{ink})$$

- Calculate net input for node Y in the output layer
- $Y_{in} = (Z_1 * w_{31}) + (Z_2 * w_{41}) + b_{w11} = 0.09101$
- Calculate final output $Y = (f(Y_{in})) = 0.5227$

Back-propagation of error (Phase II):

Step 6: Each output unit y_k ($k = 1$ to m) receives a target pattern corresponding to the input training pattern and computes the error correction term:

$$\delta_k = (t_k - y_k) f'(y_{ink})$$

The derivative $f'(y_{ink})$ can be calculated as in Section 2.3.3. On the basis of the calculated error correction term, update the change in weights and bias:

$$\Delta w_{jk} = \alpha \delta_k z_j; \quad \Delta w_{0k} = \alpha \delta_k$$

Also, send δ_k to the hidden layer backwards.

Step 7: Each hidden unit ($z_j, j = 1$ to p) sums its delta inputs from the output units:

$$\delta_{inj} = \sum_{k=1}^m \delta_k w_{jk}$$

The term δ_{inj} gets multiplied with the derivative of $f(z_{inj})$ to calculate the error term:

$$\delta_j = \delta_{inj} f'(z_{inj})$$

The derivative $f'(z_{inj})$ can be calculated as discussed in Section 2.3.3 depending on whether binary or bipolar sigmoidal function is used. On the basis of the calculated δ_j , update the change in weights and bias:

$$\Delta v_{ij} = \alpha \delta_j x_i; \quad \Delta v_{0j} = \alpha \delta_j$$

Weight and bias updation (Phase III):

Step 8: Each output unit (y_k , $k = 1$ to m) updates the bias and weights:

$$w_{jk}(\text{new}) = w_{jk}(\text{old}) + \Delta w_{jk}$$

$$w_{0k}(\text{new}) = w_{0k}(\text{old}) + \Delta w_{0k}$$

Each hidden unit (z_j , $j = 1$ to p) updates its bias and weights:

$$v_{ij}(\text{new}) = v_{ij}(\text{old}) + \Delta v_{ij}$$

$$v_{0j}(\text{new}) = v_{0j}(\text{old}) + \Delta v_{0j}$$

Step 9: Check for the stopping condition. The stopping condition may be certain number of epochs reached or when the actual output equals the target output.

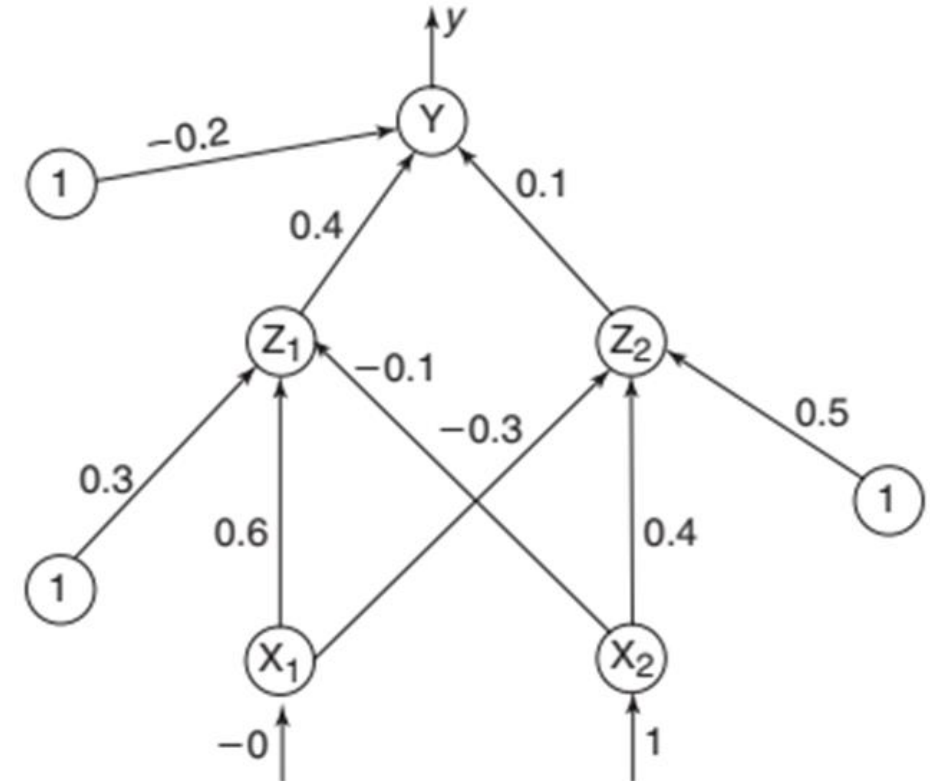
Compute the error portion δ_k

$$\delta_k = (t_k - y_k) f'(y_{ink})$$

$$f'(y_{in}) = f(y_{in})[1 - f(y_{in})]$$

- $f'(Y_{in}) = 0.5227 [1 - 0.5227]$
- $f'(Y_{in}) = 0.2495$
- $\delta_1 = (1 - 0.5227) * 0.2495$
- **$\delta_1 = 0.1191$**

Backpropagation Phase



- $\delta_1 = 0.1191$

- Find the changes in weights between hidden and output layer.

$$\Delta w_{31} = \alpha \delta_1 z_1$$

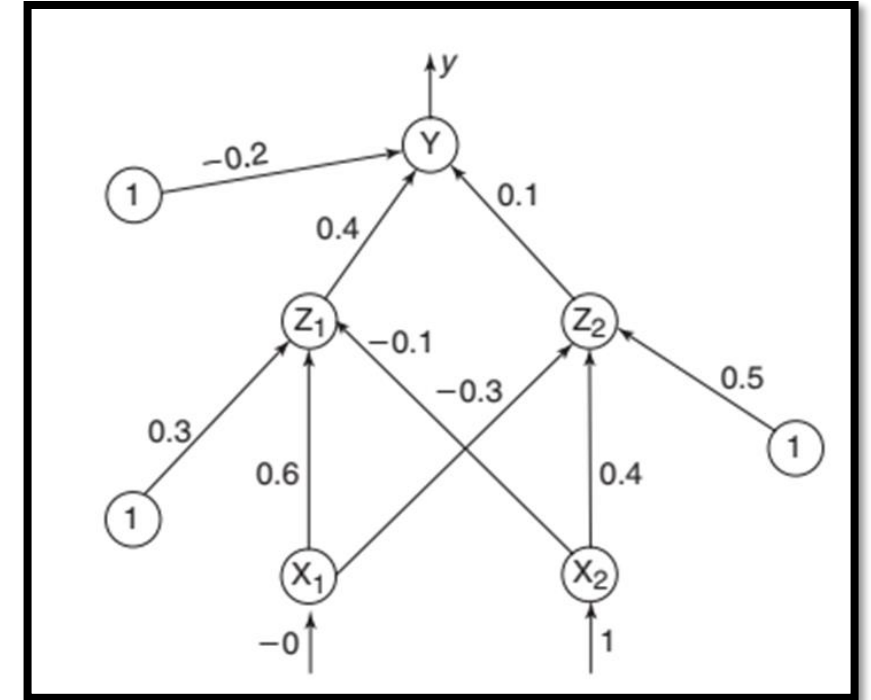
- $\Delta w_{31} = 0.25 * 0.1191 * 0.5498 = 0.0164$

- Similarly compute :

- $\Delta w_{41} = \alpha \delta_1 z_2$

- $\Delta b_{w11} = \alpha \delta_1$

Backpropagation Phase



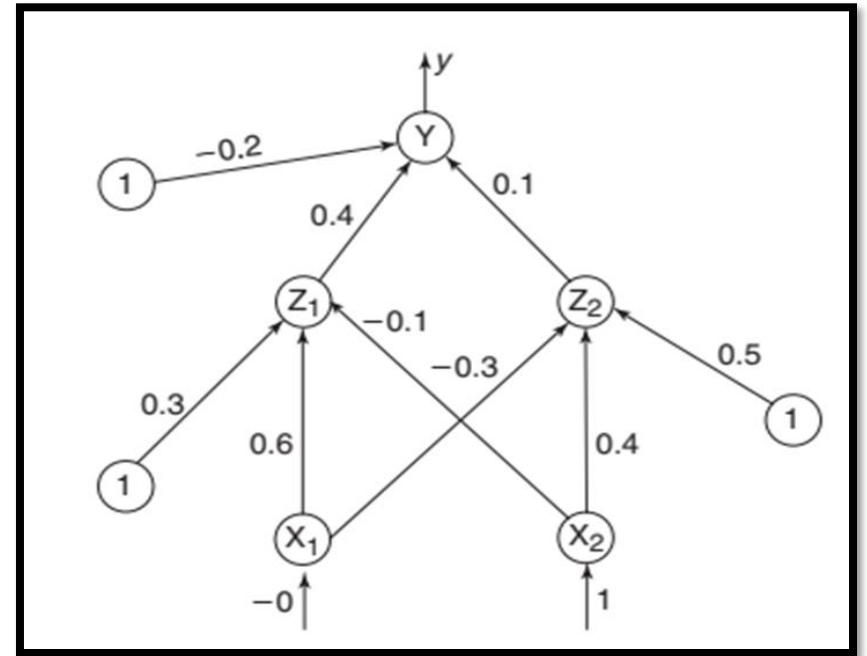
- $\Delta w_{41} = 0.02117$

- $\Delta b_{w11} = 0.02978$

Compute the error portion δ_j between input and hidden layer (from $j = 1$ to 2)

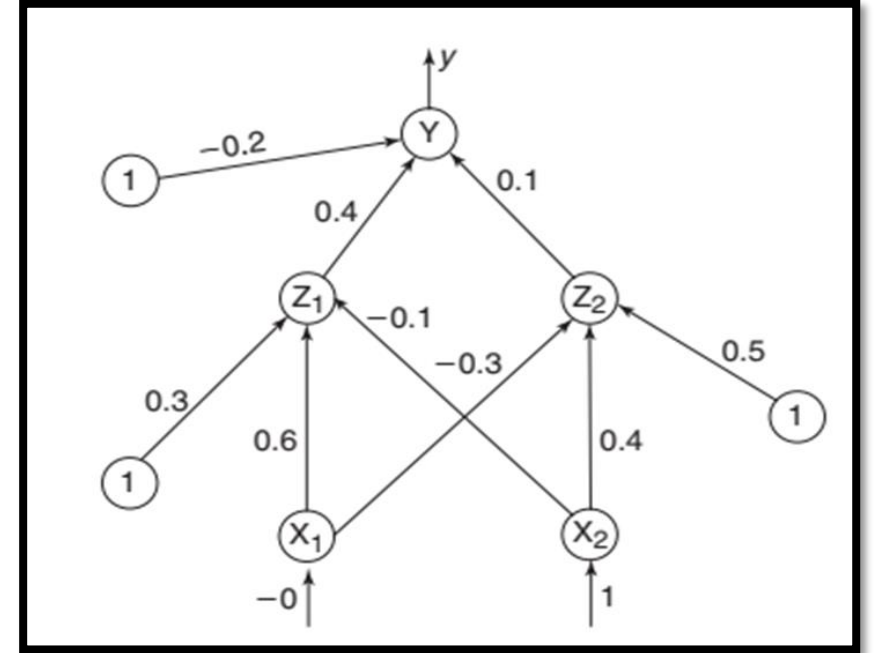
$$\delta_j = \delta_{inj} f'(z_{inj})$$

- $\delta_2 = \delta_{in2} * f'(Zin_1)$
- $\delta_2 = (0.1191 * 0.4) * 0.2475$
- $\delta_2 = 0.0118$
- $\delta_3 = \delta_{in3} * f'(Zin_2)$
- $\delta_3 = 0.01191 * 0.2055$
- $\delta_3 = 0.00245$



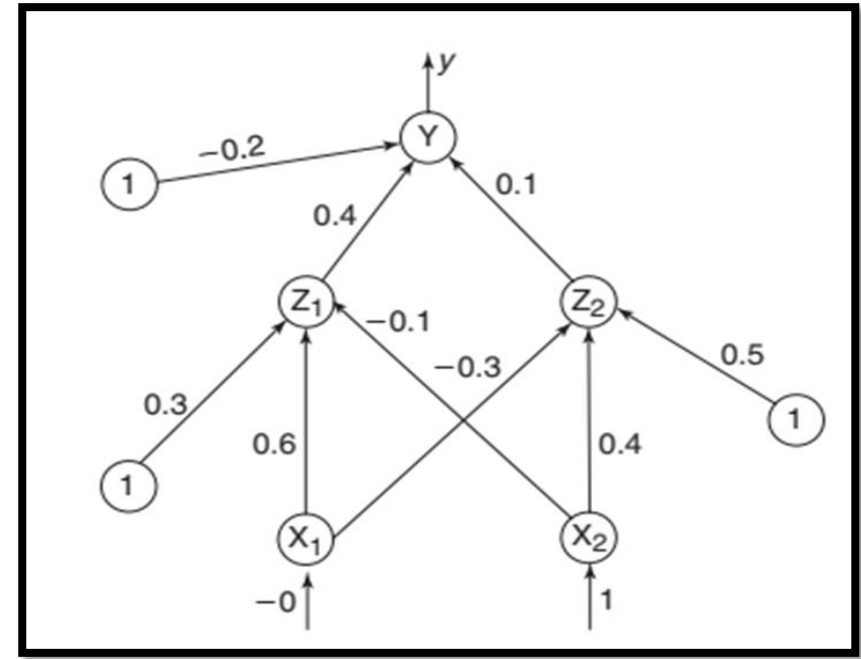
Calculate Change in weight

- $\Delta w_{11} = \alpha \delta_2 x_1$
- $\Delta w_{21} = \alpha \delta_2 x_2$
- $\Delta b w_{01} = \alpha \delta_2$
- $\Delta w_{12} = \alpha \delta_3 x_1$
- $\Delta w_{22} = \alpha \delta_3 x_2$
- $\Delta b w_{02} = \alpha \delta_3$



Update the weights of the network

- $W_{11} = W_{11_old} + \Delta w_{11}$
- $W_{12} = W_{12_old} + \Delta w_{12}$
- $W_{21} = W_{21_old} + \Delta w_{21}$
- $W_{22} = W_{22_old} + \Delta w_{22}$
- $b_{01} = bw_{01_old} + \Delta bw_{01}$
- $b_{02} = bw_{02_old} + \Delta bw_{02}$



- $W_{31} = W_{31_old} + \Delta w_{31}$
- $W_{41} = W_{41_old} + \Delta w_{41}$
- $b_{w11} = bw_{w11_old} + \Delta bw_{w11}$

- Will back-propagation learning algorithm can always converge and find proper weights for network even after enough learning?
- It will converge since it implements a gradient-descent on the error surface in the weight space, and this will roll down the error surface to the nearest minimum error and will stop.
- This becomes true only when the relation existing between the input and the output training patterns is deterministic and the error surface is deterministic.

- This is not the case in real world because the produced square-error surfaces are always at random.
- This is the stochastic nature of the back-propagation algorithm, which is purely based on the stochastic gradient-descent method.
- The BPN is a special case of stochastic approximation.
- If the BPN algorithm converges at all, then it may get stuck with local minima and may be unable to find satisfactory solutions.
- The randomness of the algorithm helps it to get out of local minima.
- The error functions may have large number of global minima because of permutations of weights that keep the network input-output function unchanged. This causes the error surfaces to have numerous troughs.

Practice Example 2

